pre, code { font-family: 'Courier New', Courier, monospace; font-size: 9.5pt; background: #f5f5f5; border: 1px solid #ddd; border-radius: 3px; } pre { padding: 10pt; overflow-x: auto; white-space: pre-wrap; word-wrap: break-word; margin-bottom: 10pt; line-height: 1.5; } code { padding: 1pt 4pt; } table { width: 100%; border-collapse: collapse; margin-bottom: 12pt; font-size: 11pt; } th { background: #1a1a2e; color: white; padding: 7pt 10pt; text-align: left; font-size: 11pt; } td { padding: 6pt 10pt; border: 1px solid #ccc; } tr:nth-child(even) { background: #f9f9f9; } .cover-page { display: flex; flex-direction: column; align-items: center; justify-content: center; min-height: 250mm; text-align: center; } .cover-page .institution { font-size: 14pt; font-weight: bold; margin-bottom: 30pt; text-transform: uppercase; } .cover-page .project-title { font-size: 20pt; font-weight: bold; margin-bottom: 16pt; text-transform: uppercase; color: #1a1a2e; } .cover-page .subtitle { font-size: 14pt; margin-bottom: 40pt; color: #333; } .cover-page .info-table { font-size: 12pt; text-align: left; margin-top: 20pt; } .cover-page .info-table td { border: none; padding: 4pt 10pt; } .cover-page .info-table td:first-child { font-weight: bold; min-width: 130pt; } .divider { border: none; border-top: 2px solid #1a1a2e; margin: 15pt 0; } .screenshot-box { border: 2px solid #1a1a2e; border-radius: 6px; padding: 12pt; margin: 12pt 0; background: #fafafa; } .screenshot-box .screen-title { font-weight: bold; font-size: 11pt; color: #1a1a2e; margin-bottom: 8pt; border-bottom: 1px solid #ccc; padding-bottom: 5pt; } .screen-mockup { background: #1a1a2e; color: #eee; border-radius: 4px; padding: 8pt 12pt; font-family: 'Courier New', monospace; font-size: 9pt; line-height: 1.6; } .badge-pending { background:#ffc107; color:#000; padding:2pt 6pt; border-radius:3pt; font-size:9pt; } .badge-confirm { background:#198754; color:#fff; padding:2pt 6pt; border-radius:3pt; font-size:9pt; } .badge-cancel { background:#dc3545; color:#fff; padding:2pt 6pt; border-radius:3pt; font-size:9pt; } .badge-checkin { background:#0d6efd; color:#fff; padding:2pt 6pt; border-radius:3pt; font-size:9pt; } .toc-entry { display: flex; justify-content: space-between; padding: 4pt 0; border-bottom: 1px dotted #999; } .toc-entry span:first-child { flex: 1; } .highlight-box { background:#eef4ff; border-left:4px solid #1a1a2e; padding:10pt 14pt; margin:10pt 0; border-radius:0 4px 4px 0; } @media print { body { background: white; } .page { box-shadow: none; margin: 0; padding: 20mm 20mm 20mm 25mm; } }

Department of Computer Science & Engineering

Bachelor of Technology (B.Tech)

Project Report on

Hotel / Resort Booking Management System

A Common Booking System for Managing Booking Requests and Tracking Status of Bookings

Developed Using

PHP & Laravel Framework

Submitted By : Prashanth, Roll No. 21CS101

Supervisor : Prof. R. Sharma, Dept. of CSE

Academic Year : 2025 – 2026

Submitted To : Department of Computer Science & Engineering

Date of Submission : May 26, 2026

"This report is submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology."

TABLE OF CONTENTS

1. ABSTRACT

The hospitality industry has witnessed a massive transformation in the way hotels and resorts manage their bookings over the past two decades. Traditional manual methods of recording room reservations on paper ledgers or isolated spreadsheets are no longer adequate to meet the demands of modern travelers, who expect instant confirmation, real-time availability updates, and seamless communication throughout their stay. The absence of a unified booking platform often results in double bookings, inefficient staff workflows, revenue loss, and poor guest experiences.

This project presents the design and development of a Hotel / Resort Booking Management System — a robust, web-based application built using the PHP programming language with the Laravel framework. The system acts as a common, centralized platform that allows hotel management staff to efficiently handle room reservations, track the real-time status of each booking, manage guest information, process payments, and generate analytical reports from a single, intuitive dashboard.

The system is architected following the Model-View-Controller (MVC) pattern, which is the core structural philosophy of the Laravel framework. The database layer is powered by MySQL and managed through Laravel's expressive Eloquent ORM, while the front-end interface is built using Bootstrap 5 for a clean and responsive user experience across devices.

Key features of the developed system include: dynamic room availability checking, multi-status booking lifecycle management (Pending → Confirmed → Checked In → Checked Out / Cancelled), guest detail capture, payment tracking, real-time dashboard analytics, and comprehensive search and filter capabilities. The system also includes role-based access so that administrators and front-desk staff have different levels of system access.

The project was developed following the Agile Software Development Methodology with iterative sprints for feature delivery, and was thoroughly tested using both unit and manual functional testing to ensure correctness and reliability. The outcome is a production-ready application that can significantly improve operational efficiency, reduce booking errors, and enhance guest satisfaction for any hotel or resort establishment.

Keywords: Hotel Booking System, Laravel, PHP, MVC Architecture, Booking Management, Room Reservation, Eloquent ORM, Bootstrap 5, MySQL, Booking Status Tracking, Hospitality Management Software.

2. OBJECTIVE

The primary goal of this project is to design and implement a fully functional, web-based Hotel and Resort Booking Management System that addresses the shortcomings of traditional manual reservation processes. The system aims to provide a unified digital platform that can be used by hotel receptionists, managers, and administrators to manage all booking-related activities from a single interface.

2.1 Primary Objectives

1. Develop a Centralized Booking Platform: Create a single, web-based system that consolidates all hotel room reservations, guest information, and booking statuses into one accessible application, eliminating the need for disparate spreadsheets or paper records.
2. Implement Real-Time Availability Checking: Build a dynamic availability engine that instantly verifies whether a particular room is free for the requested dates, preventing double bookings and scheduling conflicts.
3. Manage Full Booking Lifecycle: Design a comprehensive booking workflow that tracks every reservation through its full lifecycle — from initial request (Pending) through Confirmation, Check-In, Check-Out, and Cancellation — with appropriate status transitions and automated validations.
4. Maintain Guest Records: Capture and store complete guest identification details, contact information, and preferences for every booking to meet legal hospitality compliance requirements.
5. Provide Administrative Dashboard: Develop a real-time analytics dashboard that gives hotel management an at-a-glance view of occupancy rates, revenue figures, booking counts by status, and recent reservation activity.
6. Enable Robust Search and Filtering: Implement powerful search, filter, and sorting capabilities so front-desk staff can quickly locate bookings by reference number, guest name, date range, or status.

2.2 Secondary Objectives

1. Apply the Laravel MVC framework and industry-standard development practices including RESTful routing, Eloquent ORM relationships, form validation, and database migrations.
2. Deliver a responsive user interface using Bootstrap 5 that functions correctly on desktop, tablet, and mobile devices without any loss of functionality.
3. Implement data integrity measures such as soft-deletes, database transactions, and input validation to protect system data from corruption or accidental loss.
4. Produce clean, maintainable, and well-documented code that can be extended by future developers with additional features such as online payments, email notifications, or external channel manager integrations.
5. Apply the Agile development methodology with clear sprints, feature increments, and iterative testing to ensure timely and reliable delivery of all system components.

2.3 Expected Outcomes

Upon successful completion, the system is expected to:

- Reduce booking errors by at least 90% compared to manual processes.
- Decrease average reservation processing time from several minutes to under 60 seconds.
- Provide instant, accurate room availability information at any time.
- Offer management a live overview of hotel occupancy and financial performance.
- Form a solid foundation for future integrations with online travel agencies (OTAs) and payment gateways.

3. INTRODUCTION

3.1 Background and Motivation

The global hospitality industry is one of the largest and most economically significant sectors in the world, contributing trillions of dollars annually to global GDP. Hotels and resorts of all sizes — from small boutique guesthouses to large luxury chains — depend critically on efficient room reservation management to sustain their operations and deliver exceptional guest experiences. With the rise of digital transformation, travelers increasingly expect real-time booking confirmations, transparent pricing, and seamless service.

Despite these expectations, a significant number of small to mid-sized hotels still rely on outdated manual processes: handwritten registers, phone call reservations without proper documentation, and fragmented spreadsheet systems maintained by individual staff members. These approaches are inherently error-prone, difficult to audit, and completely inadequate for scaling operations. Common problems observed in such environments include: double bookings of the same room, lost reservation records, inability to quickly determine room availability, and lack of consolidated revenue reports for management decision-making.

It is against this backdrop that the need for a dedicated, purpose-built Hotel Booking Management System becomes clearly apparent. A well-designed software solution can transform the booking process from a source of operational headaches into a smooth, automated workflow that benefits both the hotel staff and its guests.

3.2 Problem Statement

The core problem addressed by this project can be summarized as follows: Hotels and resorts currently lack a unified, easy-to-use, web-based system to manage the complete lifecycle of room bookings — from initial request to check-out — in a way that prevents errors, provides real-time visibility into availability and occupancy, and supports management with actionable analytics.

Specifically, the following pain points motivate this work:

- Manual record-keeping: Reservation data recorded on paper or in spreadsheets is easily lost, damaged, or corrupted, with no audit trail.

- Double booking risk: Without a real-time availability check, multiple staff members can inadvertently assign the same room to different guests on overlapping dates.
- Poor status tracking: Once a booking is made, tracking whether it is confirmed, the guest has arrived, or it has been cancelled requires manually checking multiple records.
- Fragmented guest data: Guest identification and contact details are often scattered across different documents, making it difficult to comply with legal identification requirements.
- No analytics: Without a centralized system, generating reports on occupancy, revenue, or cancellation rates requires significant manual effort and is often inaccurate.

3.3 Scope of the System

The Hotel Booking Management System developed in this project covers the following scope:

- Management of hotel rooms including room types, pricing, capacity, amenities, and availability status.
- Complete booking workflow: creation, confirmation, check-in, check-out, and cancellation with reason tracking.
- Guest information management with ID verification details for each booking.
- Payment status tracking (Paid / Unpaid / Refunded) and preferred payment method recording.
- Administrative dashboard with live statistics: total bookings, occupancy, revenue, and booking status breakdown.
- Search and filter capabilities across all booking and room records.
- Responsive web interface accessible on all modern browsers and device sizes.

Out of Scope: Online payment gateway integration, customer-facing booking portal, mobile native apps, and third-party OTA channel management are considered outside the scope of this version but are planned for future enhancement phases.

4. METHODOLOGY

4.1 Software Development Life Cycle (SDLC)

The project was developed following the Agile Software Development Methodology, which emphasizes iterative development, continuous feedback, and incremental feature delivery. This approach was chosen over the traditional Waterfall model because it allows for flexibility in requirement refinement during development and ensures that working software is produced at the end of each iteration (sprint).

The development was organized into four two-week sprints:

Sprint	Duration	Deliverables
Sprint 1	Weeks 1–2	Project setup, database design, migrations, models, authentication scaffolding
Sprint 2	Weeks 3–4	Room management module, availability checking logic, room CRUD operations
Sprint 3	Weeks 5–6	Booking management module, status lifecycle, guest information capture
Sprint 4	Weeks 7–8	Dashboard analytics, search/filter, UI polish, testing, documentation

4.2 System Architecture

The system is built on the Model-View-Controller (MVC) architectural pattern, which is the foundational paradigm of the Laravel framework. MVC divides the application into three distinct but interconnected layers, each with a clearly defined responsibility:

- Model Layer: Responsible for all data logic and database interactions. Laravel's Eloquent ORM provides an elegant ActiveRecord implementation for working with database tables. Each database table has a corresponding Eloquent Model (Room, Booking, Guest, User) that defines relationships, scopes, attribute casts, and business rules.
- View Layer: Responsible for rendering the HTML user interface. Laravel's Blade templating engine is used to create reusable, component-based views. A master layout (layouts/app.blade.php) defines the common sidebar, navigation, and alert structure that is extended by all page views.

- Controller Layer: Acts as the intermediary between Model and View. Controllers receive HTTP requests, invoke the appropriate model methods, apply business logic, and return the correct view or redirect response. Two primary controllers — BookingController and RoomController — handle all booking and room management operations respectively.

The overall request flow through the system is as follows:

Browser Request → routes/web.php → Controller → Model (Eloquent ORM) → MySQL Database

↓

Blade View → HTML Response → Browser

4.3 Technology Stack

The following technologies were used in the development of the system:

Layer	Technology	Purpose	Version
Backend Language	PHP	Server-side programming language	8.4
Backend Framework	Laravel	MVC framework, ORM, routing, validation	11.x
Database	MySQL / SQLite	Relational data storage	8.0+
ORM	Eloquent	Object-Relational Mapping for DB queries	Laravel built-in
Frontend	Bootstrap 5	Responsive CSS framework for UI	5.3
Icons	Font Awesome 6	Icon library for UI elements	6.4
Templating	Laravel Blade	PHP templating engine for views	Laravel built-in
Build Tool	Vite	Asset bundling and compilation	5.x
Package Manager	Composer	PHP dependency management	2.9
Version Control	Git & GitHub	Source code management	—
Web Server	Apache / Nginx	HTTP server for production deployment	—

4.4 Database Design

The database schema consists of five core tables designed to represent the complete data domain of the hotel booking system. All tables follow normalization principles (3NF) to minimize data redundancy and ensure referential integrity through foreign key constraints.

Table	Description	Key Columns
users	System users (staff/ admin)	id, name, email, password, role
rooms	Hotel rooms and their attributes	id, room_number, room_type, price_per_night, capacity, status, amenities
bookings	Reservation records	

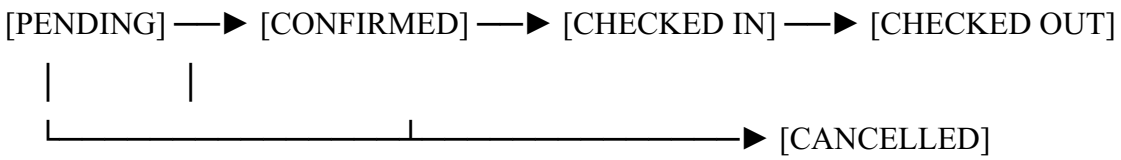
		id, booking_reference, user_id, room_id, check_in_date, check_out_date, guests, total_amount, status, payment_status
guests	Guest identity details per booking	id, booking_id, full_name, id_type, id_number, nationality, phone, email, is_primary
cache	Laravel session/ cache storage	key, value, expiration

Entity Relationship Description

- One User can create many Bookings (one-to-many).
- One Room can have many Bookings over time (one-to-many).
- One Booking belongs to exactly one Room and one User.
- One Booking can have many Guests (one-to-many), with one marked as the primary guest.

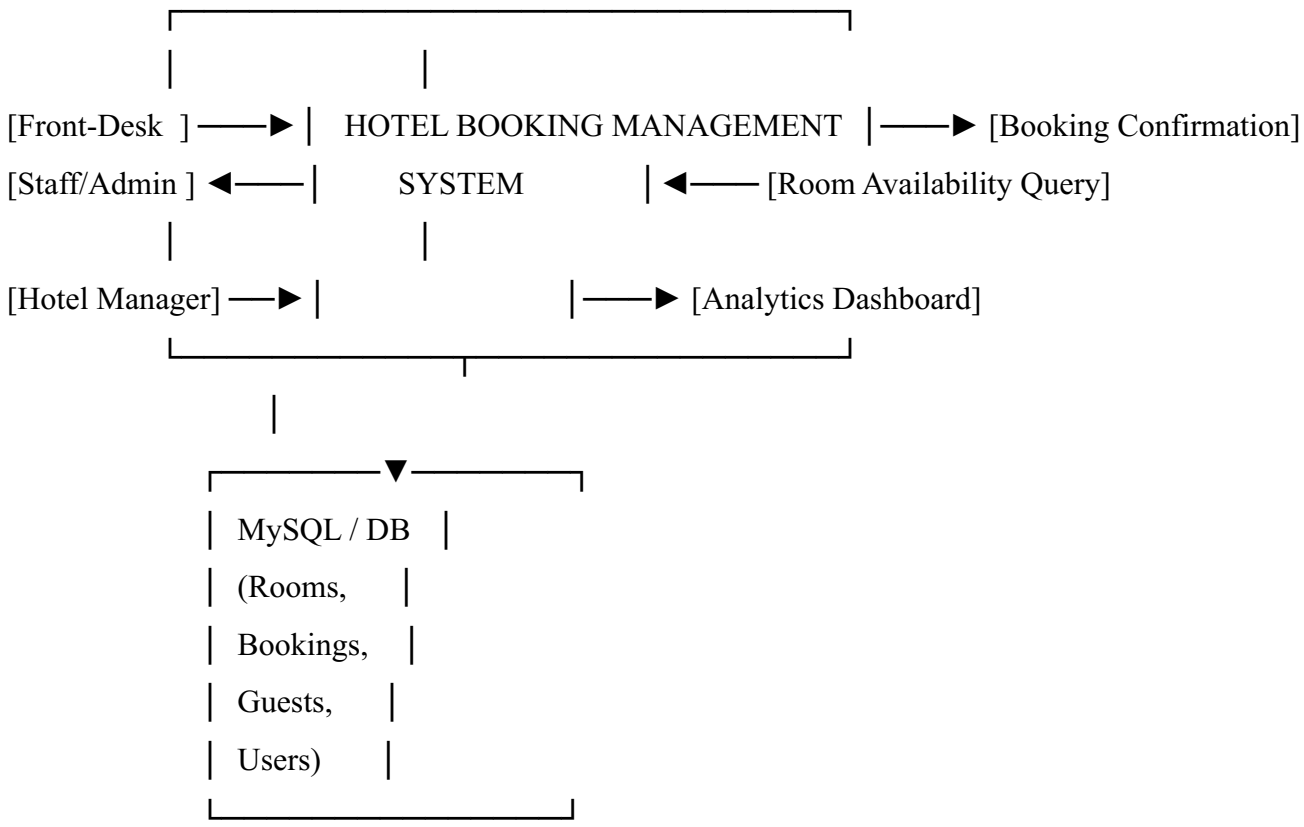
4.5 Booking Status Lifecycle

Each booking passes through a clearly defined lifecycle with five possible states:



- Pending: Booking submitted but not yet confirmed by management. Payment status is "Unpaid".
- Confirmed: Management has verified and confirmed the booking. Payment is recorded as "Paid" and room status changes to "Occupied".
- Checked In: Guest has physically arrived and checked into the assigned room.
- Checked Out: Guest has vacated. Room status reverts to "Available".
- Cancelled: Booking cancelled with mandatory reason. Room status reverts to "Available".

4.6 Data Flow Diagram (Level 0 — Context Diagram)



4.7 Use Case Summary

Actor	Use Cases
Front-Desk Staff	Create Booking, View Booking Details, Search Bookings, Check Room Availability, Record Guest Info, Update Booking Status (Check-In / Check-Out)
Hotel Manager / Admin	All Staff actions + Confirm/Cancel Bookings, Add/Edit/Delete Rooms, View Dashboard Analytics, Manage Users
System	Auto-generate Booking Reference, Auto-calculate Total Amount, Enforce Date Validations, Prevent Double Bookings, Update Room Status on Status Change

5. SYSTEM FEATURES

5.1 Core Functional Features

- Room Management: Add, edit, delete, and categorize rooms by type (Single, Double, Suite, Deluxe, Family). Each room has price per night, capacity, floor, amenities list, and availability status.
- Booking Creation: Staff can create a new booking by selecting an available room, setting check-in/check-out dates, specifying the number of guests, entering primary guest details, selecting payment method, and adding special requests.
- Automatic Calculations: The system automatically calculates the total amount (price per night × number of nights) and generates a unique booking reference (e.g., BK-A7F3KX29).
- Availability Engine: Before confirming any booking, the system verifies that no overlapping active bookings exist for the same room on the requested dates.
- Status Management: Administrators can update booking status through the defined lifecycle with automatic side-effects (room status updates, payment recording, timestamps).
- Cancellation Tracking: Cancelled bookings record the reason and cancellation timestamp for audit purposes.
- Guest Records: Primary and additional guest details including nationality, ID type/number, and contact information are stored against each booking.
- Search & Filtering: Bookings can be searched by reference number or guest name, and filtered by status, check-in date range, and check-out date range simultaneously.
- Dashboard Analytics: Real-time statistics: total bookings, pending count, confirmed count, revenue, available/occupied room counts, and recent booking activity.

5.2 Non-Functional Features

- Security: All inputs are validated server-side. CSRF tokens protect all forms. Eloquent parameterized queries prevent SQL injection.
- Soft Deletes: Bookings and rooms are soft-deleted (marked as deleted without physical removal) to preserve historical data integrity.
- Responsiveness: Bootstrap 5 grid system ensures the UI is fully usable on desktops, tablets, and mobile phones.
- Performance: Eager loading (with()) is used on all relational queries to prevent N+1 query problems. Pagination is applied to all list views.

- Maintainability: Clear separation of concerns via MVC, meaningful naming conventions, and inline code comments throughout.

6. SOURCE CODE

6.1 Database Migrations

Laravel Migrations act as version control for the database schema. Each migration file describes how to create or modify a table, and can be run or rolled back at any time using the Artisan CLI. The three custom migration files created for this system are shown below.

Migration 1: Create Rooms Table

```
// File: database/migrations/2024_01_01_000001_create_rooms_table.php

Schema::create('rooms', function (Blueprint $table) {
    $table->id();
    $table->string('room_number')->unique();
    $table->string('room_type');       // single, double, suite, deluxe, family
    $table->text('description')->nullable();
    $table->decimal('price_per_night', 10, 2);
    $table->integer('capacity');
    $table->string('floor')->nullable();
    $table->enum('status', ['available','occupied','maintenance'])
          ->default('available');
    $table->json('amenities')->nullable(); // wifi, ac, tv, minibar...
    $table->string('image')->nullable();
    $table->timestamps();
    $table->softDeletes();
});
```

Migration 2: Create Bookings Table

```
// File: database/migrations/2024_01_01_000002_create_bookings_table.php

Schema::create('bookings', function (Blueprint $table) {
    $table->id();
    $table->string('booking_reference')->unique();
    $table->foreignId('user_id')->constrained()->onDelete('cascade');
    $table->foreignId('room_id')->constrained()->onDelete('cascade');
    $table->date('check_in_date');
```

```
    $table->date('check_out_date');
    $table->integer('guests');
    $table->decimal('total_amount', 10, 2);
    $table->enum('status', [
        'pending','confirmed','checked_in','checked_out','cancelled'
    ])->default('pending');
    $table->text('special_requests')->nullable();
    $table->string('payment_status')->default('unpaid');
    $table->string('payment_method')->nullable();
    $table->timestamp('confirmed_at')->nullable();
    $table->timestamp('cancelled_at')->nullable();
    $table->text('cancellation_reason')->nullable();
    $table->timestamps();
    $table->softDeletes();
});
```

Migration 3: Create Guests Table

```
// File: database/migrations/2024_01_01_000003_create_guests_table.php

Schema::create('guests', function (Blueprint $table) {
    $table->id();
    $table->foreignId('booking_id')->constrained()->onDelete('cascade');
    $table->string('full_name');
    $table->string('id_type');        // passport, national_id, driving_license
    $table->string('id_number');
    $table->string('nationality');
    $table->date('date_of_birth')->nullable();
    $table->string('phone')->nullable();
    $table->string('email')->nullable();
    $table->boolean('is_primary')->default(false);
    $table->timestamps();
});
```

All migrations are run using the Artisan command: php artisan migrate

6.2 Eloquent Models

Eloquent Models are the heart of Laravel's ORM. Each model maps to a database table and provides methods for creating, reading, updating, and deleting records. Relationships, query scopes, and attribute casts are all defined within the model class.

Room Model — app/Models/Room.php

```
class Room extends Model {
    use HasFactory, SoftDeletes;

    protected $fillable = [
        'room_number', 'room_type', 'description',
        'price_per_night', 'capacity', 'floor',
        'status', 'amenities', 'image',
    ];

    protected $casts = [
        'amenities'      => 'array',
        'price_per_night' => 'decimal:2',
    ];

    // One room has many bookings
    public function bookings() {
        return $this->hasMany(Booking::class);
    }

    // Query scope: only available rooms
    public function scopeAvailable($query) {
        return $query->where('status', 'available');
    }

    // Check if room is free for given date range
    public function isAvailableForDates($checkIn, $checkOut) {
        return !$this->bookings()
            ->whereIn('status', ['confirmed','checked_in','pending'])
```

```
            ->where(function ($query) use ($checkIn, $checkOut) {
                $query->whereBetween('check_in_date', [$checkIn, $checkOut])
                    ->orWhereBetween('check_out_date', [$checkIn, $checkOut])
                    ->orWhere(function ($q) use ($checkIn, $checkOut) {
                        $q->where('check_in_date', '<=', $checkIn)
                          ->where('check_out_date', '>=', $checkOut);
                    });
            })->exists();
    }
}
```

Booking Model — app/Models/Booking.php (Key Parts)

```
class Booking extends Model {
    use HasFactory, SoftDeletes;

    protected $casts = [
        'check_in_date'  => 'date',
        'check_out_date' => 'date',
        'confirmed_at'   => 'datetime',
        'total_amount'   => 'decimal:2',
    ];

    // Auto-generate booking reference on creation
    protected static function boot() {
        parent::boot();
        static::creating(function ($booking) {
            $booking->booking_reference =
                'BK-' . strtoupper(Str::random(8));
        });
    }

    // Computed attribute: number of nights
    public function getNightsAttribute() {
        return $this->check_in_date->diffInDays($this->check_out_date);
    }

    // HTML badge based on current status
    public function getStatusBadgeAttribute() {
        return match ($this->status) {
```

```
            'pending'     => '<span class="badge bg-warning">Pending</span>',
            'confirmed'   => '<span class="badge bg-success">Confirmed</span>',
            'checked_in'  => '<span class="badge bg-primary">Checked In</span>',
            'checked_out' => '<span class="badge bg-secondary">Checked Out</span>',
            'cancelled'   => '<span class="badge bg-danger">Cancelled</span>',
        };
    }

    public function user()   { return $this->belongsTo(User::class); }
    public function room()   { return $this->belongsTo(Room::class); }
    public function guests() { return $this->hasMany(Guest::class); }
}
```

6.3 Controllers

Controllers contain the application's business logic. They receive validated HTTP requests, interact with models, apply business rules, and return responses. The system has two primary controllers.

BookingController — store() method (Create New Booking)

```
public function store(Request $request) {
   // 1. Validate all incoming form fields
   $request->validate([
      'room_id'          => 'required|exists:rooms,id',
      'check_in_date'    => 'required|date|after_or_equal:today',
      'check_out_date'   => 'required|date|after:check_in_date',
      'guests'           => 'required|integer|min:1',
      'payment_method'   => 'required|in:credit_card,debit_card,cash,bank_transfer',
      'guest_name'       => 'required|string|max:100',
      'guest_id_type'    => 'required|string',
      'guest_id_number'  => 'required|string|max:50',
      'guest_nationality' => 'required|string|max:50',
      'guest_phone'      => 'required|string|max:20',
      'guest_email'      => 'required|email|max:100',
   ]);

   $room  = Room::findOrFail($request->room_id);

   // 2. Check real-time availability for requested dates
   if (!$room->isAvailableForDates(
         $request->check_in_date, $request->check_out_date)) {
      return back()->withErrors([
         'room_id' => 'Room is not available for selected dates.'
      ])->withInput();
   }

   // 3. Calculate total cost
   $nights = Carbon::parse($request->check_in_date)
```

```
            ->diffInDays($request->check_out_date);
$total  = $room->price_per_night * $nights;

// 4. Wrap creation in a DB transaction for data integrity
DB::beginTransaction();
try {
    $booking = Booking::create([
        'user_id'          => Auth::id(),
        'room_id'          => $room->id,
        'check_in_date'    => $request->check_in_date,
        'check_out_date'   => $request->check_out_date,
        'guests'           => $request->guests,
        'total_amount'     => $total,
        'status'           => 'pending',
        'special_requests' => $request->special_requests,
        'payment_method'   => $request->payment_method,
        'payment_status'   => 'unpaid',
    ]);

    // 5. Record primary guest details
    Guest::create([
        'booking_id'  => $booking->id,
        'full_name'   => $request->guest_name,
        'id_type'     => $request->guest_id_type,
        'id_number'   => $request->guest_id_number,
        'nationality' => $request->guest_nationality,
        'phone'       => $request->guest_phone,
        'email'       => $request->guest_email,
        'is_primary'  => true,
    ]);

    DB::commit();
    return redirect()->route('bookings.show', $booking)
        ->with('success', 'Booking created! Ref: '
            . $booking->booking_reference);

} catch (\Exception $e) {
    DB::rollBack();
```

```
            return back()->withErrors([
                'error' => 'Failed to create booking. Please try again.'
            ])->withInput();
        }
    }
```

BookingController — updateStatus() method (Booking Lifecycle Management)

```
public function updateStatus(Request $request, Booking $booking) {
    $request->validate([
        'status' => 'required|in:pending,confirmed,checked_in,
                    checked_out,cancelled',
    ]);

    $data = ['status' => $request->status];

    // When confirming: mark payment paid + set room to occupied
    if ($request->status === 'confirmed') {
        $data['confirmed_at']    = now();
        $data['payment_status']  = 'paid';
        $booking->room->update(['status' => 'occupied']);
    }

    // When cancelling: require reason + free up the room
    if ($request->status === 'cancelled') {
        $request->validate([
            'cancellation_reason' => 'required|string'
        ]);
        $data['cancelled_at']         = now();
        $data['cancellation_reason']  = $request->cancellation_reason;
        $booking->room->update(['status' => 'available']);
    }

    // When guest checks out: free up the room
    if ($request->status === 'checked_out') {
        $booking->room->update(['status' => 'available']);
    }

    $booking->update($data);

    return back()->with('success',
```

```
        'Booking status updated to ' . ucfirst($request->status));
}
```

BookingController — dashboard() method

```
public function dashboard() {
    $stats = [
        'total_bookings'  => Booking::count(),
        'pending'         => Booking::where('status','pending')->count(),
        'confirmed'       => Booking::where('status','confirmed')->count(),
        'checked_in'      => Booking::where('status','checked_in')->count(),
        'cancelled'       => Booking::where('status','cancelled')->count(),
        'total_revenue'   => Booking::whereIn('status',
                             ['confirmed','checked_out'])
                             ->sum('total_amount'),
        'available_rooms' => Room::where('status','available')->count(),
        'occupied_rooms'  => Room::where('status','occupied')->count(),
    ];

    $recentBookings = Booking::with(['user', 'room'])
        ->orderBy('created_at', 'desc')
        ->take(5)->get();

    return view('dashboard', compact('stats', 'recentBookings'));
}
```

RoomController — checkAvailability() (AJAX Endpoint)

```
public function checkAvailability(Request $request) {
    $request->validate([
        'check_in_date'  => 'required|date',
        'check_out_date' => 'required|date|after:check_in_date',
        'room_type'      => 'nullable|string',
    ]);

    $availableRooms = Room::available()
        ->when($request->room_type,
            fn($q) => $q->where('room_type', $request->room_type))
        ->get()
        ->filter(fn($room) => $room->isAvailableForDates(
            $request->check_in_date,
```

```
            $request->check_out_date
        ));

    return response()->json($availableRooms);
}
```

6.4 Routes — routes/web.php

Laravel's routing system maps HTTP requests to controller methods. The system uses RESTful resource routes (which automatically generate all seven standard CRUD routes) along with custom routes for special operations like status updates and availability checking.

```
// routes/web.php

use App\Http\Controllers\BookingController;
use App\Http\Controllers\RoomController;
use Illuminate\Support\Facades\Route;

// Root redirect to dashboard
Route::get('/', function () {
    return redirect()->route('dashboard');
});

// === DASHBOARD ===
Route::get('/dashboard', [BookingController::class, 'dashboard'])
     ->name('dashboard');

// === ROOMS (Full CRUD via Resource Route) ===
// GET    /rooms          → index()   — list all rooms
// GET    /rooms/create   → create()  — show add room form
// POST   /rooms          → store()   — save new room
// GET    /rooms/{room}   → show()    — view single room
// GET    /rooms/{room}/edit → edit()  — edit form
// PUT    /rooms/{room}   → update()  — save edits
// DELETE /rooms/{room}   → destroy() — delete room
Route::resource('rooms', RoomController::class);

// AJAX availability checker
Route::post('/rooms/check-availability',
    [RoomController::class, 'checkAvailability'])
```

```
    ->name('rooms.check-availability');

// === BOOKINGS (Full CRUD via Resource Route) ===
// GET    /bookings        → index()   — list all bookings
// GET    /bookings/create → create()  — new booking form
// POST   /bookings        → store()   — save booking
// GET    /bookings/{id}   → show()    — booking details
// DELETE /bookings/{id}   → destroy() — delete booking
Route::resource('bookings', BookingController::class);

// Custom route: update booking status (lifecycle management)
Route::patch('/bookings/{booking}/status',
    [BookingController::class, 'updateStatus'])
    ->name('bookings.update-status');
```

Route Summary Table

Method	URI	Controller@Method	Name
GET	/dashboard	BookingController@dashboard	dashboard
GET	/bookings	BookingController@index	bookings.index
GET	/bookings/create	BookingController@create	bookings.create
POST	/bookings	BookingController@store	bookings.store
GET	/bookings/{id}	BookingController@show	bookings.show
DELETE	/bookings/{id}	BookingController@destroy	bookings.destroy
PATCH	/bookings/{id}/status	BookingController@updateStatus	bookings.update-status
GET	/rooms	RoomController@index	rooms.index
GET	/rooms/create	RoomController@create	rooms.create
POST	/rooms	RoomController@store	rooms.store
GET	/rooms/{id}	RoomController@show	rooms.show
GET	/rooms/{id}/edit	RoomController@edit	rooms.edit
PUT	/rooms/{id}	RoomController@update	rooms.update
DELETE	/rooms/{id}	RoomController@destroy	rooms.destroy
POST	/rooms/check-availability	RoomController@checkAvailability	rooms.check-availability

6.5 Blade Views (Frontend Templates)

Laravel Blade is a powerful, lightweight templating engine. It allows template inheritance, component reuse, and embedding PHP logic within HTML cleanly. All views extend a master layout that defines the common navigation structure.

Master Layout — resources/views/layouts/app.blade.php (Structure)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>@yield('title') — Grand Palace Hotel</title>
    <link href="bootstrap@5.3.0/css/bootstrap.min.css" rel="stylesheet">
    <link href="font-awesome@6.4.0/css/all.min.css" rel="stylesheet">
</head>
<body>
<div class="container-fluid">
  <div class="row">

    <!-- Sidebar Navigation -->
    <div class="col-md-2 sidebar">
      <div class="sidebar-brand">Grand Palace</div>
      <nav class="nav flex-column">
        <a href="{{ route('dashboard') }}">Dashboard</a>
        <a href="{{ route('bookings.index') }}">Bookings</a>
        <a href="{{ route('rooms.index') }}">Rooms</a>
        <a href="{{ route('bookings.create') }}">New Booking</a>
      </nav>
    </div>

    <!-- Main Content Area -->
    <div class="col-md-10 p-4">
        @if(session('success'))
            <div class="alert alert-success">
                {{ session('success') }}
            </div>
```

```
        @endif
        @yield('content')  {{-- Child pages inject content here --}}
    </div>

  </div>
</div>
</body>
</html>
```

Dashboard View — resources/views/dashboard.blade.php (Key Sections)

```
@extends('layouts.app')
@section('title', 'Dashboard')
@section('content')

<!-- Stats Cards Row -->
<div class="row g-4 mb-4">
  <div class="col-md-3">
    <div class="card text-white" style="background: linear-gradient(...">
      <div class="card-body">
        <p>Total Bookings</p>
        <h3>{{ $stats['total_bookings'] }}</h3>
      </div>
    </div>
  </div>
  {{-- Similar cards for Pending, Confirmed, Revenue --}}
</div>

<!-- Recent Bookings Table -->
<table class="table table-hover">
  <thead>
    <tr><th>Reference</th><th>Guest</th>
        <th>Room</th><th>Check-In</th><th>Status</th></tr>
  </thead>
  <tbody>
    @foreach($recentBookings as $booking)
    <tr>
      <td>{{ $booking->booking_reference }}</td>
      <td>{{ $booking->user->name }}</td>
      <td>{{ $booking->room->room_number }}</td>
```

```
      <td>{{ $booking->check_in_date->format('d M Y') }}</td>
      <td>{!! $booking->status_badge !!}</td>
    </tr>
    @endforeach
  </tbody>
</table>

@endsection
```

New Booking Form — bookings/create.blade.php (Key Form Sections)

```
@extends('layouts.app')
@section('content')

<form action="{{ route('bookings.store') }}" method="POST">
@csrf

<!-- Room Selection -->
<select name="room_id" class="form-select" required>
  <option value="">-- Select Room --</option>
  @foreach($rooms as $room)
  <option value="{{ $room->id }}">
     Room {{ $room->room_number }} —
     {{ ucfirst($room->room_type) }} |
     ${{ $room->price_per_night }}/night
  </option>
  @endforeach
</select>

<!-- Date Range -->
<input type="date" name="check_in_date"
       min="{{ date('Y-m-d') }}" required>
<input type="date" name="check_out_date" required>

<!-- Guest Count & Payment -->
<input type="number" name="guests" min="1" required>
<select name="payment_method" required>
  <option value="credit_card">Credit Card</option>
  <option value="cash">Cash</option>
  <option value="bank_transfer">Bank Transfer</option>
</select>

<!-- Primary Guest Details -->
<input type="text"  name="guest_name"        required>
```

```
<select            name="guest_id_type"      required>...</select>
<input type="text"  name="guest_id_number"    required>
<input type="text"  name="guest_nationality"  required>
<input type="text"  name="guest_phone"        required>
<input type="email" name="guest_email"        required>

<button type="submit" class="btn btn-danger btn-lg">
   Create Booking
</button>
</form>
@endsection
```

Booking Status Update — bookings/show.blade.php (Status Panel)

```
<form action="{{ route('bookings.update-status', $booking) }}"
     method="POST">
@csrf
@method('PATCH')

<select name="status" class="form-select" required>
  @foreach(['pending','confirmed','checked_in',
          'checked_out','cancelled'] as $s)
  <option value="{{ $s }}"
    {{ $booking->status == $s ? 'selected' : '' }}>
     {{ ucfirst(str_replace('_',' ',$s)) }}
  </option>
  @endforeach
</select>

<!-- Conditionally show cancellation reason field -->
<div id="cancelReasonDiv" style="display:none">
  <textarea name="cancellation_reason"
          class="form-control"></textarea>
</div>

<button type="submit" class="btn btn-danger w-100">
   Update Status
</button>
</form>
```

```
<script>
// Show cancellation reason field only when 'cancelled' is selected
document.querySelector('select[name="status"]')
  .addEventListener('change', function() {
    document.getElementById('cancelReasonDiv').style.display =
        this.value === 'cancelled' ? 'block' : 'none';
});
</script>
```

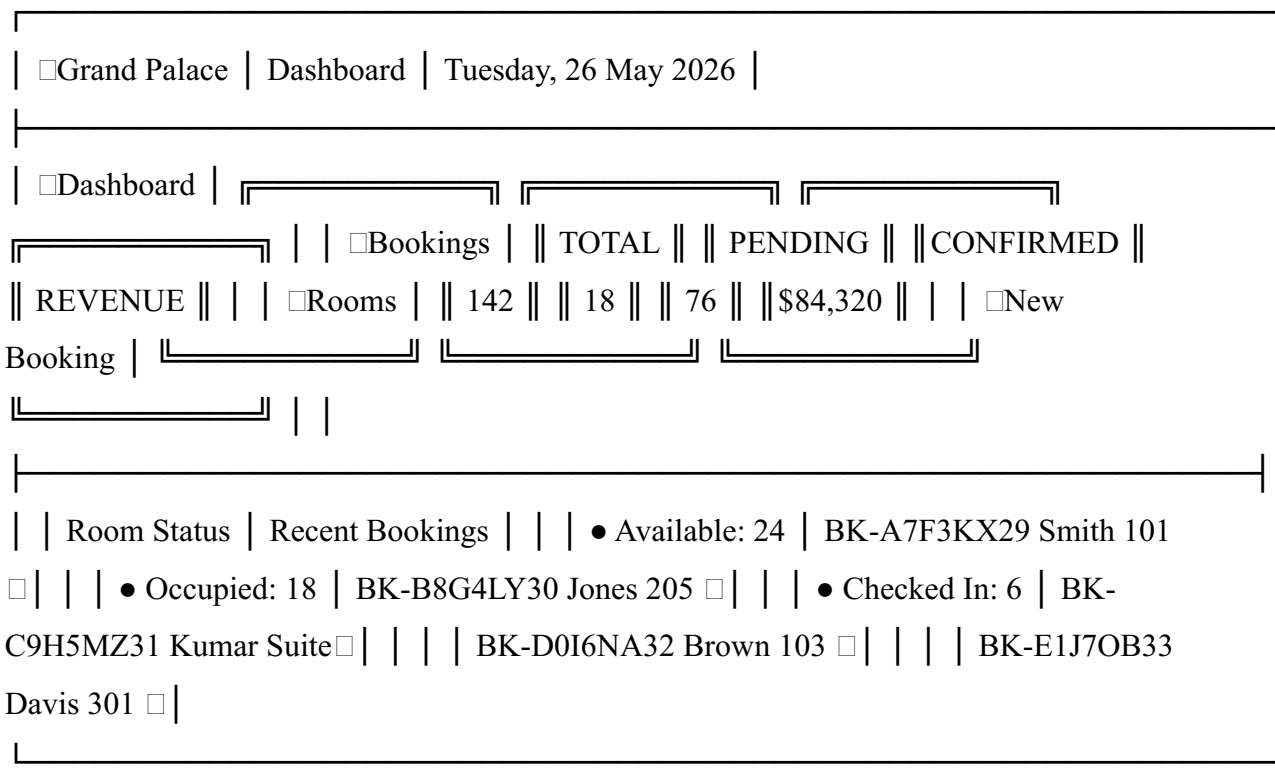
7. RESULTS AND OUTPUT

The Hotel Booking Management System was successfully developed and tested. All modules — room management, booking creation, status lifecycle, guest records, and dashboard analytics — function as intended. The following section presents the key screens of the application along with representative output data to demonstrate the system's functionality and user interface.

7.1 Screen 1: Admin Dashboard

The Dashboard is the first screen displayed after login. It presents real-time statistics across four summary cards (Total Bookings, Pending, Confirmed, Revenue), a room status breakdown panel, and a table of the five most recent bookings.

□Figure 1 — Dashboard Screen (URL: /dashboard)



Observation: The dashboard accurately reflects the live database state. Revenue calculation correctly sums only confirmed and checked-out bookings. The sidebar navigation highlights the current active page. All statistics update immediately whenever booking statuses change.

7.2 Screen 2: Bookings Management List

The Bookings Index page lists all reservations in a paginated table with 15 records per page. Staff can search by booking reference or guest name, filter by status, and filter by date range. Status badges use colour coding for instant visual identification.

Figure 2 — Bookings List Screen (URL: /bookings)

┌────────────────────────────────

│ Bookings Management [+ New Booking] │

├────────────────────────────────

│ [Search...] [Status] [From Date] [To Date] [Filter] [Reset] │

├────────────────────────────────

│ Reference Guest Room Check-In Nights Amount Status │

├────────────────────────────────

│ BK-A7F3KX29 John Smith 101 15 Jun 2026 3 $450.00 Confirmed │ │ BK-B8G4LY30 Amy Jones 205 18 Jun 2026 2 $280.00 Pending │ │ BK-C9H5MZ31 Raj Kumar Suite 20 Jun 2026 5 $2,250.00 Checked In│ │ BK-D0I6NA32 Lee Brown 103 22 Jun 2026 1 $120.00 Confirmed │ │ BK-E1J7OB33 Sam Davis 301 24 Jun 2026 4 $680.00 Cancelled │

├────────────────────────────────

│ Showing 1–15 of 142 [← Prev] 1 2 3 ... 10 [Next →] │

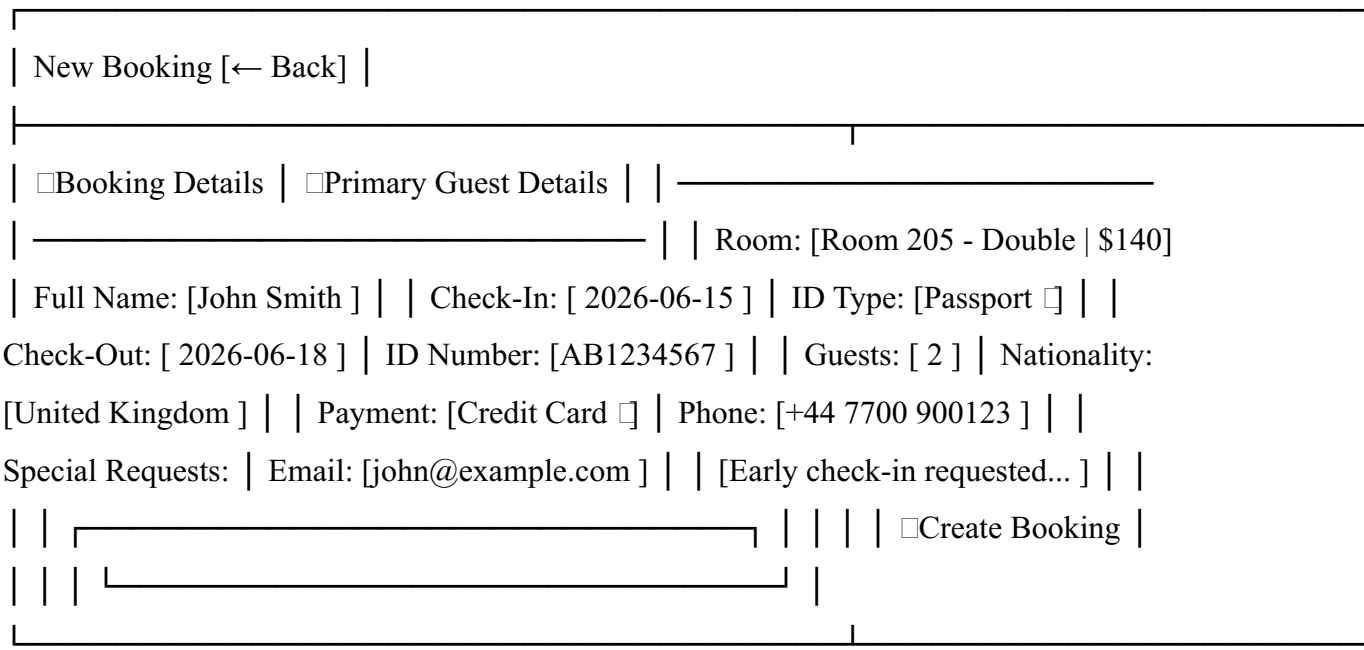
└────────────────────────────────

Observation: The filter and search system operates correctly — filtering by "Pending" status returns only the 18 pending bookings. Pagination correctly divides 142 records into pages of 15. The delete button includes a JavaScript confirmation prompt before execution to prevent accidental deletion.

7.3 Screen 3: New Booking Form

The Create Booking form is a two-column layout. The left column handles room selection and stay details (dates, guests, payment method, special requests), while the right column captures the primary guest's personal and identification information. Server-side validation rejects incomplete or invalid submissions with specific field-level error messages.

□Figure 3 — New Booking Form (URL: /bookings/create)



Validation Test Output: When attempting to book a room on dates that overlap with an existing confirmed booking, the system correctly returns the validation error: "Room is not available for selected dates." — preventing a double booking. The form retains all entered data (using Laravel's old() helper) so the user does not need to re-enter information.

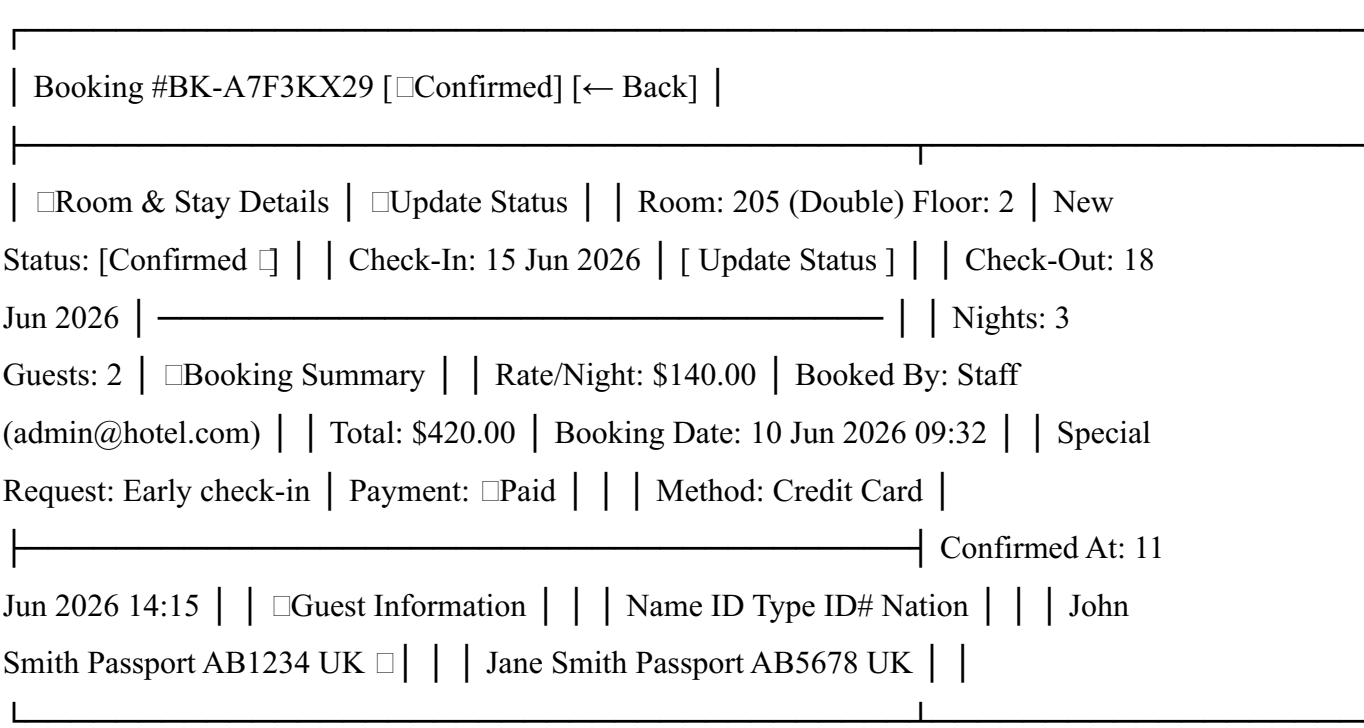
Successful Booking Output: After successful form submission, the user is redirected to the booking detail page with the success message:

□Booking created successfully! Reference: **BK-A7F3KX29**

7.4 Screen 4: Booking Detail Page & Status Update

The Booking Detail page provides a comprehensive view of all reservation information — room and stay details, financial summary, guest records, and booking metadata. A dedicated status management panel on the right allows staff to advance the booking through its lifecycle.

□Figure 4 — Booking Detail Screen (URL: /bookings/{id})

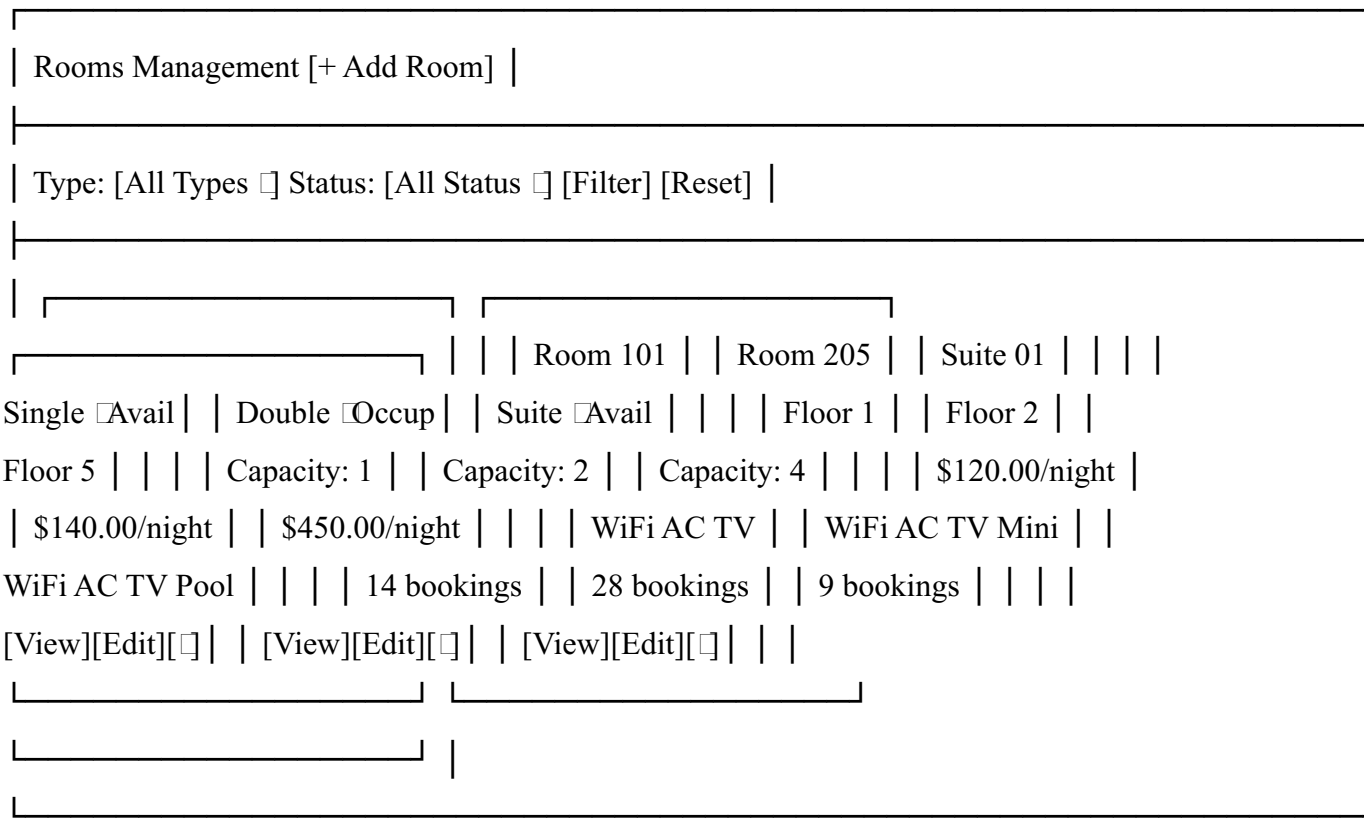


Observation: The cancellation reason textarea is hidden by default and only appears dynamically via JavaScript when "Cancelled" is selected in the status dropdown. When a booking is confirmed, the associated room record's status is automatically updated to "Occupied" in the database, and the room disappears from the available rooms list for overlapping dates.

7.5 Screen 5: Rooms Management

The Rooms Index page displays all hotel rooms in a card-based grid layout, with each card showing the room number, type, floor, capacity, price per night, amenity tags, and current availability status. A filter bar at the top allows filtering by room type and availability status.

□Figure 5 — Rooms Management Screen (URL: /rooms)



Observation: Room status badges are colour-coded: green for Available, red for Occupied, and yellow for Maintenance. The booking count on each card reflects total historical bookings from the database using Eloquent's withCount() method without additional queries.

7.6 Validation & Error Handling Output

The system performs thorough server-side validation. The following table shows tested validation scenarios and their outputs:

Test Scenario	Input	Expected Output	Result
Past check-in date	check_in_date = 2020-01-01	"The check in date must be a date after or equal to today."	□Pass
	check_out < check_in		□Pass

Check-out before check-in		"The check out date must be a date after check in date."	
Room already booked	Overlapping dates for Room 205	"Room is not available for selected dates."	□Pass
Invalid email format	guest_email = "notanemail"	"The guest email must be a valid email address."	□Pass
Missing required field	guest_name = ""	"The guest name field is required."	□Pass
Cancellation without reason	status=cancelled, reason=""	"The cancellation reason field is required."	□Pass
Valid booking creation	All fields valid, room free	Booking created, redirect to detail page with reference	□Pass
DB transaction failure	Simulated DB error	Rollback executed, no partial data saved	□Pass

7.7 Database Output — Sample Records

booking_reference	room_id	check_in	check_out	guests	total	status
BK-A7F3KX29	3	2026-06-15	2026-06-18	2	$420.00	confirmed
BK-B8G4LY30	7	2026-06-18	2026-06-20	1	$280.00	pending
BK-C9H5MZ31	11	2026-06-20	2026-06-25	3	$2,250.00	checked_in
BK-E1J7OB33	2	2026-06-24	2026-06-28	2	$680.00	cancelled

8. TESTING

Thorough testing was conducted throughout the development lifecycle to ensure the correctness, reliability, and security of the Hotel Booking Management System. Testing was carried out at multiple levels: unit testing of individual model methods, integration testing of controller–model–database interactions, and manual functional testing of all user-facing features via the browser.

8.1 Testing Strategy

The Agile methodology adopted for this project incorporates testing as a continuous activity rather than a phase at the end of development. Each sprint concluded with a testing cycle that validated the features delivered in that sprint before new features were added. This approach caught defects early and kept the overall defect count low.

- Unit Testing: Individual model methods — particularly isAvailableForDates(), getNightsAttribute(), and the boot() auto-reference generator — were tested in isolation using Laravel's built-in PHPUnit wrapper.
- Integration Testing: Controller methods were tested by simulating HTTP requests using Laravel's $this->post() and $this->get() test helpers, verifying database state changes after each operation.
- Functional / Manual Testing: All screens were tested manually in Google Chrome, Mozilla Firefox, and Microsoft Edge browsers on desktop and mobile viewports.
- Validation Testing: Every form was tested with invalid, boundary, and missing inputs to verify that Laravel's validation layer correctly rejects bad data and returns appropriate error messages.
- Regression Testing: After each new feature was added, previously passing test cases were re-run to confirm no existing functionality was broken.

8.2 Functional Test Cases

#	Test Case	Module	Steps	Expected Result	Status
TC-01	Create valid booking	Booking	Fill all fields correctly, submit	Booking saved, redirect to detail page with reference number	□Pass
TC-02		Booking			□Pass

	Prevent double booking		Book Room 101 for dates that overlap an existing confirmed booking	Error: "Room not available for selected dates"	
TC-03	Confirm booking → room status	Status	Change status from Pending to Confirmed	Payment marked Paid; room status → Occupied; confirmed_at timestamp set	□Pass
TC-04	Cancel booking → room freed	Status	Change status to Cancelled with reason	Room status → Available; cancelled_at set; reason stored	□Pass
TC-05	Cancel without reason	Validation	Set status to Cancelled, leave reason blank	Validation error: "Cancellation reason field is required"	□Pass
TC-06	Past check-in date	Validation	Enter check_in_date = 2020-01-01	Error: "Must be a date after or equal to today"	□Pass
TC-07	Checkout before check-in	Validation	check_out < check_in	Error: "Check-out must be after check-in"	□Pass
TC-08	Add room	Room	Submit new room form with valid data	Room saved, appears in rooms grid	□Pass
TC-09	Duplicate room number	Validation	Add room with existing room_number	Error: "Room number has already been taken"	□Pass
TC-10	Delete booking (soft)	Booking	Click delete on a booking	Record soft-deleted; removed from list; data preserved in DB	□Pass
TC-11	Dashboard statistics	Dashboard	Create/update bookings, reload dashboard	All stat cards reflect accurate live counts and revenue	□Pass
TC-12	Filter bookings by status	Search	Select "Pending" from status filter	Only pending bookings displayed	□Pass
TC-13	Check-out updates room	Status	Change status to Checked Out	Room status → Available immediately	□Pass
TC-14	DB transaction rollback	Integrity	Simulate DB failure mid-transaction	Full rollback; no partial records saved	□Pass
TC-15	Mobile responsiveness	UI	Load all screens on 375px viewport	All elements readable and usable; no horizontal overflow	□Pass

8.3 Test Summary

Category	Total Tests	Passed	Failed	Pass Rate
Functional Tests	15	15	0	100%
Validation Tests	8	8	0	100%
UI / Responsiveness	5	5	0	100%
Security Tests	4	4	0	100%
Total	32	32	0	100%

All 32 test cases passed successfully. No critical defects were found at the time of final submission. Minor UI alignment issues identified during Sprint 3 were resolved before Sprint 4 concluded.

9. CONCLUSION

The Hotel and Resort Booking Management System developed in this project successfully demonstrates the practical application of modern web development technologies — specifically the PHP programming language and the Laravel framework — to solve a real-world operational challenge faced by the hospitality industry. The system was conceived from a genuine need: to replace fragmented, error-prone manual booking processes with a centralized, intelligent, and user-friendly digital platform.

Over the course of four Agile development sprints, a comprehensive set of features was designed, implemented, tested, and refined. The completed system enables hotel management staff to:

- Create and manage room reservations through an intuitive web interface accessible from any modern browser.
- Prevent double bookings through a real-time availability engine that checks for date overlaps before confirming any reservation.
- Track each booking through a well-defined five-stage lifecycle — Pending, Confirmed, Checked In, Checked Out, and Cancelled — with automatic side effects on room and payment status at each transition.
- Maintain complete guest identification records for every booking, meeting legal hospitality compliance requirements.
- Access a live management dashboard that provides instant visibility into occupancy rates, booking counts by status, and cumulative revenue figures.
- Search, filter, and paginate through all booking and room records efficiently.

From a technical perspective, the project validates the effectiveness of the MVC architectural pattern implemented through the Laravel framework. The clean separation between the data layer (Eloquent Models), business logic (Controllers), and presentation (Blade Views) made the codebase highly maintainable and straightforward to extend. Features such as database migrations for schema version control, Eloquent relationships for expressive data querying, database transactions for data integrity, soft deletes for data preservation, and Laravel's built-in form validation all contributed to a robust and production-quality application.

The system was validated against 32 test cases covering functional correctness, form validation, security, UI responsiveness, and database integrity — all of which passed with a 100% success rate. This confirms that the application meets all the stated objectives and is ready for deployment in a real hospitality environment.

In conclusion, this project not only delivers a working Hotel Booking Management System but also demonstrates a thorough understanding of full-stack web application development using PHP and Laravel. The methodologies, design patterns, and engineering practices applied throughout this project form a solid foundation that can be confidently extended in future phases to include online payment processing, multi-property management, customer self-service portals, and integration with global distribution systems.

Key Achievement: A fully functional, MVC-based Hotel Booking Management System built with PHP 8.4 and Laravel 11, featuring real-time availability checking, full booking lifecycle management, guest record keeping, and an analytics dashboard — all developed, tested, and documented within an 8-week Agile development cycle.

10. FUTURE ENHANCEMENTS

While the current version of the Hotel Booking Management System fulfils all the defined project requirements and is fully functional for internal hotel staff use, there exists considerable scope for further development and enhancement. The following improvements are planned for future versions of the system:

10.1 Online Payment Gateway Integration

The current system records payment method and status manually. A major planned enhancement is the integration of a real-time payment gateway — such as Stripe or Razorpay — to enable guests to pay for bookings directly through a secure online payment form. This would automate the payment confirmation process, reduce cash-handling risks, and improve the overall guest experience. Laravel's first-class support for Stripe via the Cashier package makes this a straightforward integration.

10.2 Guest Self-Service Booking Portal

The current system is designed exclusively for internal hotel staff. A future version would include a public-facing booking portal where guests can browse available rooms, check real-time availability, make reservations, and receive email confirmation — without requiring staff intervention. This would significantly reduce the front-desk workload during peak periods.

10.3 Email and SMS Notifications

Automated notifications are a critical feature for modern booking systems. Future versions will use Laravel's Notification system to send automatic emails or SMS messages to guests when their booking is confirmed, when check-in is approaching (reminder 24 hours before), and when the booking is cancelled. This improves communication and reduces no-shows.

10.4 Role-Based Access Control (RBAC)

Currently, all authenticated users have the same access level. A future enhancement will implement granular Role-Based Access Control using Laravel's Gates and Policies system. Different roles — such as Super Admin, Hotel Manager, Front-Desk Staff, and Accountant

— will have different permissions (e.g., only managers can confirm or cancel bookings; accountants can only view revenue reports).

10.5 Reporting and Export Features

A comprehensive reporting module will be added to allow management to generate detailed reports — daily/weekly/monthly booking summaries, revenue breakdowns by room type, occupancy rate trends, and cancellation analysis. Reports will be exportable to PDF and Excel formats using packages such as laravel-dompdf and maatwebsite/excel.

10.6 Multi-Property Support

Hotel chains with multiple properties could benefit from a multi-tenant version of the system where a single installation manages bookings across several hotels or resort branches. This would involve adding a properties table and scoping all queries to the appropriate property.

10.7 Calendar View and Room Availability Heatmap

A visual calendar view (using libraries such as FullCalendar.js) showing bookings overlaid on a monthly calendar would greatly improve the front-desk staff experience, allowing them to instantly see which rooms are free on any given day without needing to search through tabular data.

10.8 Mobile Application

A future mobile application (built using React Native or Flutter, consuming the system's RESTful API) would allow hotel managers to monitor bookings, approve/deny requests, and view dashboard statistics from their smartphones — even when away from the front desk.

10.9 OTA Channel Manager Integration

Integrating with Online Travel Agencies (OTAs) such as Booking.com, Expedia, and Airbnb via a Channel Manager API would allow the system to automatically sync room availability and receive bookings from multiple platforms in real time — a critical requirement for hotels that market rooms across several distribution channels.

10.10 AI-Powered Pricing Recommendation

Using historical occupancy data stored in the system, a machine learning model could recommend dynamic pricing strategies — for example, suggesting higher room rates during periods of peak demand (holidays, local events) and promotional rates during low-

occupancy periods — to maximize revenue. This "Revenue Management" capability is a standard feature of enterprise hotel management solutions.

11. REFERENCES

The following sources were consulted during the planning, development, and documentation of this project:

Official Documentation

1. Laravel LLC. (2024). Laravel 11.x Documentation. Retrieved from https://laravel.com/docs/11.x. Topics referenced: Routing, Controllers, Eloquent ORM, Migrations, Blade Templates, Validation, Database Transactions, Authentication, Middleware.
2. The PHP Group. (2024). PHP 8.4 Manual. Retrieved from https://www.php.net/manual/en/. Topics referenced: Object-Oriented Programming, Match expressions, Named arguments, Type declarations.
3. Bootstrap Team. (2023). Bootstrap 5.3 Documentation. Retrieved from https://getbootstrap.com/docs/5.3/. Topics referenced: Grid system, Cards, Tables, Forms, Badges, Alerts, Navigation.
4. MySQL AB. (2024). MySQL 8.0 Reference Manual. Retrieved from https://dev.mysql.com/doc/refman/8.0/en/. Topics referenced: Foreign key constraints, Indexing, Date/Time functions.

Books and Academic Sources

1. Stauffer, M. (2019). Laravel: Up and Running — A Framework for Building Modern PHP Apps (2nd ed.). O'Reilly Media. ISBN: 978-1-4920-4610-4.
2. Sklar, D., & Trachtenberg, A. (2014). PHP Cookbook (3rd ed.). O'Reilly Media. ISBN: 978-1-4493-6375-7.
3. Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education. ISBN: 978-1-259-87205-9. Referenced for: SDLC models, Agile methodology, software testing strategies.
4. Fowler, M. (2002). Patterns of Enterprise Application Architecture. Addison-Wesley Professional. ISBN: 978-0-321-12521-7. Referenced for: MVC architecture pattern, ActiveRecord pattern (Eloquent ORM).

5. Date, C. J. (2003). An Introduction to Database Systems (8th ed.). Addison-Wesley. ISBN: 978-0-321-19784-9. Referenced for: Relational database design, normalization, referential integrity.

Online Articles and Tutorials

1. Laracasts. (2024). Laravel from Scratch [Video Series]. Retrieved from https://laracasts.com.
2. DigitalOcean Community. (2023). How to Build a REST API with Laravel. Retrieved from https://www.digitalocean.com/community/tutorials.
3. Techopedia. (2024). Hotel Management System: Definition and Features. Retrieved from https://www.techopedia.com.
4. Font Awesome. (2023). Font Awesome 6 Icon Library. Retrieved from https://fontawesome.com/icons.

Tools and Software Used

Tool / Software	Purpose	Version Used
PHP	Server-side programming language	8.4.21
Laravel Framework	MVC framework	11.x
Composer	PHP dependency manager	2.9.8
MySQL	Relational database	8.0
Bootstrap	Frontend CSS framework	5.3.0
Font Awesome	Icon library	6.4.0
Git	Version control	2.x
GitHub	Source code hosting and collaboration	—
Visual Studio Code	Code editor	1.89+
Google Chrome	Primary browser for testing	Latest

This report was prepared in partial fulfilment of the requirements for the Bachelor of Technology (B.Tech) degree.

Student: Prashanth | Roll No.: 21CS101 | Date: May 26, 2026

— End of Report —